

Module 2: Introduction to Computer Graphics and Visual Computing

20 Guided Examples • 20 Practice Exercises

Course: CS0045 – Computer Graphics and Visual Computing

Companion: Module 2 Student Guide

Toolchain: VS Code + MSYS2 UCRT64 GCC + FreeGLUT

How to Use This Manual

This manual has three parts:

- Part A — 20 worked Examples (Ex01–Ex20). Each is a complete, compiled, ready-to-run program demonstrating one concept at a time: points, lines, stippling, filled primitives, and color. Type or copy each one into your SOURCE CODE\M2 folder, compile it (Ctrl+Shift+B), and run it (F5) before moving to the next.
- Part B — 20 Practice Exercises (Q01–Q20), split into Easy (Q01–Q07), Medium (Q08–Q14), and Hard (Q15–Q20). Read the task and hints, then write the program YOURSELF before checking Part C.
- Part C — the full Answer Key for every exercise in Part B, with complete, compiled, working code.

Setup reminder

Use the shared SOURCE CODE\ VS Code workspace described in the Module 1 Student Guide. Compile with Ctrl+Shift+B ("Build OpenGL Program"), then run with F5 or from the terminal. All programs in this manual use the same cross-platform header block shown below.

```
#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;
```

Every program in this manual has been compiled and verified with g++ against FreeGLUT.

Part A — Guided Examples (Ex01–Ex20)

Example 01 — Points with Per-Vertex Color

Concept: GL_POINTS; a different glColor3f before each glVertex2f colors each point independently.

Expected output: Four colored points (white, magenta, blue, green) arranged in a square.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;
/*
 *      Example 01 - Points with Per-Vertex Color
 *      -----
 *      Concept: GL_POINTS renders one point per vertex. Calling glColor3f * before each
 *      glVertex2f gives each point its own color. */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glPointSize(6.0f);    glBegin(GL_POINTS);        glColor3f(1.0f,
1.0f, 1.0f); glVertex2f(-0.5f, -0.5f); // white    glColor3f(1.0f,
0.0f, 1.0f); glVertex2f(-0.5f,  0.5f); // magenta    glColor3f(0.0f,
0.0f, 1.0f); glVertex2f( 0.5f,  0.5f); // blue    glColor3f(0.0f,
1.0f, 0.0f); glVertex2f( 0.5f, -0.5f); // green    glEnd();

    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc,
argv);    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex01 - Colored Points (GL_POINTS)");
    glutDisplayFunc(display);    glutMainLoop();
    return 0;
}
```

Example 02 — GL_LINES (Unconnected Segments)

Concept: Vertices grouped in pairs; each pair is one independent segment.

Expected output: Two separate vertical line segments, left and right.

```
#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;

/*
 *      Example 02 - GL_LINES (Unconnected Segments)
 *      -----
 *      Concept: OpenGL draws a line segment for each group of TWO
 *      vertices. * With n vertices, OpenGL renders n/2 segments; an odd
 *      leftover vertex * is ignored.
 */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f);

    glBegin(GL_LINES);
        glVertex2f(-0.5f, -0.5f);        glVertex2f(-0.5f, 0.5f);
// segment 1 (vertical, left)        glVertex2f( 0.5f, 0.5f);
glVertex2f( 0.5f, -0.5f);    // segment 2 (vertical, right)
glEnd();

    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc, argv);
glutInitWindowSize(600, 600);    glutCreateWindow("Ex02 -
Unconnected Lines (GL_LINES)");
glutDisplayFunc(display);    glutMainLoop();
    return 0;
}
```

Example 03 — GL_LINE_STRIP (Connected, Open)

Concept: Consecutive vertices connected in a chain; the shape stays open.

Expected output: An open, three-sided box outline (missing the bottom edge).

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;

/*
 *   Example 03 - GL_LINE_STRIP (Connected, Open)
 *   -----
 *   Concept: OpenGL renders a segment between vertex 1-2, 2-3, 3-4,
 *   and * so on - a connected chain that does NOT close back to the start.
 */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f);

    glBegin(GL_LINE_STRIP);
    glVertex2f(-0.5f,      -0.5f);
    glVertex2f(-0.5f,       0.5f);
    glVertex2f( 0.5f,       0.5f);
    glVertex2f( 0.5f,      -0.5f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc,
argv);    glutInitWindowSize(600, 600);
glutCreateWindow("Ex03 - Line Strip (GL_LINE_STRIP)");
glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 04 — GL_LINE_LOOP (Connected, Closed)

Concept: Same as GL_LINE_STRIP, plus a closing segment back to the first vertex.

Expected output: A fully closed square outline.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else

```

```

#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;
/*
 *      Example 04 - GL_LINE_LOOP (Connected, Closed)
 *      -----
 *      Concept: Behaves like GL_LINE_STRIP, but OpenGL adds one
extra * closing segment between the LAST and FIRST vertices. */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f);

    glBegin(GL_LINE_LOOP);
    glVertex2f(-0.5f,      -0.5f);
    glVertex2f(-0.5f,      0.5f);
    glVertex2f( 0.5f,      0.5f);
    glVertex2f( 0.5f,     -0.5f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc,
argv);    glutInitWindowSize(600, 600);
glutCreateWindow("Ex04 - Line Loop (GL_LINE_LOOP)");
glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 05 — glLineWidth

Concept: Comparing three line widths (1.0, 4.0, 8.0) side by side.

Expected output: Three horizontal lines of increasing thickness.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;

```

```

/*
 *   Example 05 - glLineWidth
 *   -----
 *   Concept: void glLineWidth(GLfloat width) sets the fixed width of
 *   lines. The default is 1.0; width must be greater than 0.0. * This example draws
 *   three lines with three different widths so * students can compare them directly. */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f);

    glLineWidth(1.0f);
    glBegin(GL_LINES);          glVertex2f(-
0.7f, 0.5f);          glVertex2f(0.7f,
0.5f);      glEnd();

    glLineWidth(4.0f);
    glBegin(GL_LINES);          glVertex2f(-
0.7f, 0.0f);          glVertex2f(0.7f,
0.0f);      glEnd();

    glLineWidth(8.0f);
    glBegin(GL_LINES);          glVertex2f(-
0.7f, -0.5f);          glVertex2f(0.7f, -
0.5f);      glEnd();

    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc,
argv);    glutInitWindowSize(600, 600);
glutCreateWindow("Ex05 - glLineWidth Comparison");
glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 06 — Line Stippling: Dashed Line

Concept: glEnable(GL_LINE_STIPPLE) + glLineStipple(1, 0x00FF).

Expected output: A single horizontal dashed line.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;
/*
 *      Example 06 - Line Stippling: Dashed Line
 *      -----
 *      Concept: glLineStipple(repeatFactor, pattern) breaks a solid line
 *      into a repeating pattern of dots/dashes. Pattern 0x00FF is 8 bits
 *      "on" followed by 8 bits "off". You must glEnable(GL_LINE_STIPPLE) * before
 *      drawing, and it's good practice to glDisable it afterward. */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f);

    glEnable(GL_LINE_STIPPLE);
    glLineWidth(2.0f);
    glLineStipple(1, 0x00FF);    // 8-on / 8-off, repeated once per bit

    glBegin(GL_LINE_STRIP);
    glVertex2f(-0.7f, 0.0f);
    glVertex2f( 0.7f, 0.0f);    glEnd();

    glDisable(GL_LINE_STIPPLE);
    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 400);
    glutCreateWindow("Ex06 - Dashed Line (0x00FF)");
    glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 07 — Line Stippling: Dotted Line + repeatFactor Concept:

Pattern 0xAAAA at repeatFactor 1 vs. 3.

Expected output: Two dotted lines; the bottom one has longer, more stretched dots.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;
/*
 *      Example 07 - Line Stippling: Dotted Line with repeatFactor
 *      -----
 *      Concept: Pattern 0xAAAA alternates a single "on" bit with a single * "off" bit,
 *      producing fine dots. Increasing repeatFactor stretches * every bit in the pattern,
 *      making each dot/gap longer. */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.2f, 0.8f, 1.0f);

    glEnable(GL_LINE_STIPPLE);
    glLineWidth(2.0f);

    glLineStipple(1, 0xAAAA); // fine dots
    glBegin(GL_LINE_STRIP);    glVertex2f(-
0.7f, 0.3f);    glVertex2f( 0.7f,
0.3f);    glEnd();

    glLineStipple(3, 0xAAAA); // same pattern, stretched 3x
    glBegin(GL_LINE_STRIP);    glVertex2f(-0.7f, -0.3f);
    glVertex2f( 0.7f, -0.3f);    glEnd();

    glDisable(GL_LINE_STIPPLE);
    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc, argv);
    glutInitWindowSize(600, 400);    glutCreateWindow("Ex07 -
Dotted Line, repeatFactor 1 vs 3");
    glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 08 — GL_TRIANGLES

Concept: One filled triangle per group of three vertices.

Expected output: A single solid white triangle.

```
#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;

/*
 *      Example 08 - GL_TRIANGLES
 *      -----
 *      Concept: OpenGL renders one filled triangle for each group
of * THREE vertices. With n vertices, OpenGL renders n/3
triangles. */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f);

    glBegin(GL_TRIANGLES);
    glVertex2f(-0.5f,    -0.5f);
    glVertex2f( 0.5f,    -0.5f);
    glVertex2f( 0.0f,     0.5f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex08 - Filled Triangle (GL_TRIANGLES)");
    glutDisplayFunc(display);    glutMainLoop();
    return 0;
}
```

Example 09 — GL_POLYGON

Concept: A single filled convex n-gon from an ordered vertex list.

Expected output: A solid white filled hexagon.

```
#define GL_SILENCE_DEPRECATED
```

```
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;
/*
 *      Example 09 - GL_POLYGON
 *      -----
 *      Concept: GL_POLYGON draws a single filled convex n-gon. OpenGL * renders an n-
sided polygon where n is the number of vertices * specified; if n < 3, OpenGL renders
nothing. */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f);

    glBegin(GL_POLYGON);
    glVertex2f(-0.4f, 0.1f);
    glVertex2f(0.1f, 0.1f);
    glVertex2f(0.2f, 0.3f);
    glVertex2f(0.1f, 0.6f);
    glVertex2f(-0.4f, 0.6f);
    glVertex2f(-0.5f, 0.3f);    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex09 - Filled Hexagon (GL_POLYGON)");
    glutDisplayFunc(display);    glutMainLoop();
    return 0;
}
```

Example 10 — GL_TRIANGLE_STRIP

Concept: A chain of triangles sharing edges.

Expected output: A solid quadrilateral-like shape formed from two connected triangles.

```
#define GL_SILENCE_DEPRECATED
```

```

#ifdef __APPLE__

#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;

/*
 *      Example 10 - GL_TRIANGLE_STRIP
 *      -----
 *      Concept: Draws a sequence of triangles that share edges.
 *      OpenGL * renders a triangle using vertices 1-2-3, then another
 *      using * 2-3-4, then 3-4-5, and so on. */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f);

    glBegin(GL_TRIANGLE_STRIP);
    glVertex2f(-0.5f,      -0.5f);
    glVertex2f( 0.5f,      -0.5f);
    glVertex2f( 0.0f,       0.5f);
    glVertex2f( 0.7f,       0.5f);
    glEnd();

    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc, argv);
glutInitWindowSize(600, 600);    glutCreateWindow("Ex10 -
Triangle Strip (GL_TRIANGLE_STRIP)");
glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 11 — GL_TRIANGLE_FAN

Concept: A fan of triangles sharing the first vertex.

Expected output: A solid fan/wedge shape.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else

```

```
#include <GL/glut.h>
```

```
#endif
#include <iostream> using
namespace std;
/*
 *      Example 11 - GL_TRIANGLE_FAN
 *      -----
 *      Concept: Draws a fan of triangles that share edges AND share
the * first vertex specified. With vertices v0..v5, OpenGL renders
 * triangles (v0,v1,v2), (v0,v2,v3), (v0,v3,v4), (v0,v4,v5). */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f);

    glBegin(GL_TRIANGLE_FAN);          glVertex2f( 0.0f, -
0.5f); // v0 - shared center          glVertex2f( 0.5f,
0.5f); // v1          glVertex2f( 0.0f,  0.5f); // v2
glVertex2f(-0.5f,  0.5f); // v3      glEnd();

    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc, argv);
glutInitWindowSize(600, 600);    glutCreateWindow("Ex11 -
Triangle Fan (GL_TRIANGLE_FAN)");
glutDisplayFunc(display);    glutMainLoop();
    return 0;
}
```

Example 12 — GL_QUADS

Concept: Two independent quadrilaterals in one glBegin block.

Expected output: A rectangle and a small square, unconnected.

```
#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;
```

```

/*
 *   Example 12 - GL_QUADS
 *   -----
 *   Concept: Draws individual convex quadrilaterals. OpenGL renders a * quad for each
group of FOUR vertices - one glBegin/glEnd block can * contain several independent
quads. */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f);

    glBegin(GL_QUADS);
        // quad 1 - top rectangle
    glVertex2f(-0.2f, 0.2f);          glVertex2f(-
0.5f, -0.2f);          glVertex2f(0.5f, -0.2f);
    glVertex2f(0.2f, 0.2f);

        // quad 2 - small square, upper right
    glVertex2f(0.6f, 0.4f);
    glVertex2f(0.0f, 0.4f);
    glVertex2f(0.0f, 0.6f);
    glVertex2f(0.6f, 0.6f);          glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Ex12 - Two Independent Quads (GL_QUADS)");
    glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 13 — GL_QUAD_STRIP

Concept: A chain of quads sharing edges; vertices alternate bottom/top.

Expected output: A single connected, irregularly shaped filled ribbon.

```

#define GL_SILENCE_DEPRECATED
#ifdef __APPLE__
#include <GLUT/glut.h>
#else

```

```

#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;
/*
 *      Example 13 - GL_QUAD_STRIP
 *      -----
 *      Concept: Draws a sequence of quadrilaterals that share edges. With
 *      vertices v0..v5, OpenGL renders a quad using (v0,v1,v3,v2), then * another using
 *      (v2,v3,v5,v4). Vertices are supplied in pairs * (bottom, top) as you move across the
 *      strip. */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(1.0f, 1.0f, 1.0f);

    glBegin(GL_QUAD_STRIP);
    glVertex2f(-0.5f, -0.5f); // v0
    glVertex2f(-0.5f,  0.0f); // v1
    glVertex2f( 0.0f, -0.5f); // v2
    glVertex2f( 0.0f,  0.0f); // v3
    glVertex2f( 0.5f, -0.3f); // v4
    glVertex2f( 0.5f,  0.3f); // v5    glEnd();

    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc,
argv);    glutInitWindowSize(600, 600);
glutCreateWindow("Ex13 - Quad Strip (GL_QUAD_STRIP)");
glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 14 — glColor3f (RGBA / Float Color)

Concept: Setting color using the 0.0-1.0 float range.

Expected output: A solid blue horizontal line.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else

```

```

#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;
/*
 *      Example 14 - glColor3f (RGBA / Float Color)
 *      -----
 *      Concept: In RGBA mode, glColor3f(r, g, b) maps available red,
 *      green, and blue intensities onto the range [0.0, 1.0], where 0.0 * means the
 *      component is absent and 1.0 is fully saturated. */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glColor3f(0.0f, 0.0f, 1.0f); // pure blue
    glLineWidth(2.0f);    glBegin(GL_LINES);
    glVertex2f(-0.8f, 0.0f);    glVertex2f(
0.8f, 0.0f);    glEnd();

    glFlush();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 400);
    glutCreateWindow("Ex14 - glColor3f");
    glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 15 — glColor3ub (RGBA / Unsigned Byte Color)

Concept: Setting color using the 0-255 integer range.

Expected output: A solid leafy-green filled triangle.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;

```

```

/*
 *   Example 15 - glColor3ub (RGBA / Unsigned Byte Color)
 *   -----
 *   Concept: glColor3ub(r, g, b) sets color using the 0-255 integer * range instead
of 0.0-1.0 floats - exactly like the RGB values you * would pick in an image editor or
color picker. */

void display() {    glClearColor(GL_COLOR_BUFFER_BIT);

    glColor3ub(92, 181, 100);    // a leafy green, chosen like a color picker
glBegin(GL_TRIANGLES);        glVertex2f(-0.5f, -0.5f);        glVertex2f( 0.5f, -0.5f);
glVertex2f( 0.0f,  0.5f);    glEnd();

    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc, argv);
glutInitWindowSize(600, 600);
    glutCreateWindow("Ex15 - glColor3ub (0-255 Range)");
glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 16 — glColor4f and Alpha Blending

Concept: glColor4f + glEnable(GL_BLEND) + glBlendFunc.

Expected output: Two overlapping semi-transparent quads (red and blue) blending in the overlap region.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;
/*
 * Example 16 - glColor4f and Alpha Blending
 * -----

```

```

* Concept: glColor4f(r, g, b, alpha) adds a fourth component that * controls
opacity: 1.0 is fully opaque, 0.0 is fully transparent. * Alpha only has a visible
effect once blending is turned on with * glEnable(GL_BLEND) and a blend function is
chosen. */

```

```

void display() {
    glClearColor(0.05f, 0.05f, 0.05f, 1.0f);    glClear(GL_COLOR_BUFFER_BIT);

    glEnable(GL_BLEND);    glBlendFunc(GL_SRC_ALPHA,
GL_ONE_MINUS_SRC_ALPHA);

    glColor4f(1.0f, 0.0f, 0.0f, 0.6f); // 60% opaque red    glBegin(GL_QUADS);
        glVertex2f(-0.5f, -0.3f);    glVertex2f(
0.1f, -0.3f);    glVertex2f( 0.1f,  0.3f);
glVertex2f(-0.5f,  0.3f);    glEnd();

    glColor4f(0.0f, 0.4f, 1.0f, 0.6f); // 60% opaque blue, overlapping
glBegin(GL_QUADS);
        glVertex2f(-0.1f, -0.3f);    glVertex2f(
0.5f, -0.3f);    glVertex2f( 0.5f,  0.3f);
glVertex2f(-0.1f,  0.3f);    glEnd();

    glDisable(GL_BLEND);    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc, argv);
glutInitWindowSize(600, 400);    glutCreateWindow("Ex16 - Alpha Blending
(glColor4f)");    glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 17 — Per-Vertex Color Interpolation on a Line

Concept: Different endpoint colors are interpolated across the line.

Expected output: A thick line that fades from green to red.

```
#define GL_SILENCE_DEPRECATED
```

```

#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;
/*
 *      Example 17 - Per-Vertex Color Interpolation on a Line
 *      -----
 *      Concept: When two endpoints of a line have DIFFERENT colors, * OpenGL smoothly
interpolates the color along the line during * rasterization.
 */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(10.0f);

    glBegin(GL_LINES);
    glColor3ub(92, 181, 100); // green
    glVertex2f(-0.8f, 0.0f);
    glColor3f(1.0f, 0.0f, 0.0f); // red
    glVertex2f( 0.8f, 0.0f);    glEnd();

    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc, argv);
glutInitWindowSize(600, 400);    glutCreateWindow("Ex17 -
Gradient Line (Per-Vertex Color)");
glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 18 — Per-Vertex Color Interpolation on a Triangle

Concept: Three different vertex colors blend across the triangle's interior.

Expected output: A triangle smoothly blending green, red, and blue.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else

```

```

#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;
/*
 *      Example 18 - Per-Vertex Color Interpolation on a Triangle
 *      -----
 *      Concept: Giving each of a triangle's three vertices a different * color produces
a smoothly blended fill across the whole interior. */

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    glBegin(GL_TRIANGLES);          glColor3ub(92, 181, 100);
    glVertex2f(-0.5f, -0.5f); // green      glColor3f(1.0f, 0.0f,
    0.0f); glVertex2f( 0.5f, -0.5f); // red      glColor3f(0.0f, 0.0f,
    1.0f); glVertex2f( 0.0f,  0.5f); // blue      glEnd();

    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc, argv);
glutInitWindowSize(600, 600);    glutCreateWindow("Ex18 - Gradient
Triangle (Per-Vertex Color)");    glutDisplayFunc(display);
glutMainLoop();
    return 0;
}

```

Example 19 — A Gallery of Stipple Patterns

Concept: Combining `glLineWidth`, `glLineStipple`, and color across several lines.

Expected output: Four stacked lines, each with a different dash/dot style and color.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>
#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;

/*

```

```

*      Example 19 - A Gallery of Stipple Patterns
*      -----
*      Concept: Combines glLineWidth, glLineStipple, and per-line color to
*      show several dash/dot styles stacked in one window, so students can * compare
*      pattern + repeatFactor + color together. */

void drawStippledLine(float y, GLint factor, GLushort pattern, float r, float g, float b)
{
    glColor3f(r, g, b);
    glLineStipple(factor, pattern);
    glBegin(GL_LINE_STRIP);
    glVertex2f(-0.8f, y);
    glVertex2f( 0.8f, y);    glEnd();
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);
    glLineWidth(3.0f);    glEnable(GL_LINE_STIPPLE);

    drawStippledLine( 0.6f, 1, 0x00FF, 1.0f, 1.0f, 1.0f); // white dashes
    drawStippledLine( 0.2f, 1, 0x0C0F, 1.0f, 0.6f, 0.0f); // orange dash-dot
    drawStippledLine(-0.2f, 1, 0xAAAA, 0.2f, 0.8f, 1.0f); // cyan fine dots
    drawStippledLine(-0.6f, 2, 0xAAAA, 1.0f, 0.2f, 0.6f); // pink stretched dots

    glDisable(GL_LINE_STIPPLE);
    glFlush();
}

int main(int argc, char** argv) {    glutInit(&argc,
argv);    glutInitWindowSize(700, 500);
glutCreateWindow("Ex19 - Stipple Pattern Gallery");
glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Example 20 — Capstone: Primitives + Color Composition

Concept: GL_QUADS, GL_TRIANGLE_FAN with per-vertex color, GL_QUAD_STRIP, and stippling combined.

Expected output: A small scene: a gradient sun, layered mountains, ground, and a dashed horizon line.

```

#define GL_SILENCE_DEPRECATION
#ifdef __APPLE__
#include <GLUT/glut.h>

```



```

#else
#include <GL/glut.h>
#endif
#include <iostream> using
namespace std;
/*
 *      Example 20 - Capstone: Primitives + Color Composition
 *      -----
 *      Concept: Combines nearly every idea from this module into one small
 *      scene: GL_QUADS for the ground, GL_TRIANGLE_FAN with per-vertex * color for a
 *      sun, GL_QUAD_STRIP for a mountain ridge, and a dashed * horizon line using
 *      glLineStipple. */
#include <cmath>

void drawSun() {
    const int segments = 40;
    glBegin(GL_TRIANGLE_FAN);          glColor3f(1.0f,
0.9f, 0.2f);          glVertex2f(0.55f, 0.55f);
        for (int i = 0; i <= segments; i++) {          float angle = (float)i /
segments * 2.0f * 3.14159265f;          glColor3f(1.0f, 0.5f, 0.0f);
glVertex2f(0.55f + 0.18f * cosf(angle), 0.55f + 0.18f * sinf(angle));
        }
    glEnd();
}

void
        drawMountains()
    {
glBegin(GL_QUAD_STRIP);          glColor3f(0.35f,
0.25f, 0.45f);          glVertex2f(-1.0f, -0.2f);
glColor3f(0.55f,          0.45f,          0.65f);
glVertex2f(-1.0f,          0.1f);
glColor3f(0.35f,          0.25f,          0.45f);
glVertex2f(-0.3f,          -0.2f);
glColor3f(0.55f,          0.45f,          0.65f);
glVertex2f(-0.3f,          0.3f);
glColor3f(0.35f,          0.25f,          0.45f);
glVertex2f(          0.4f,          -0.2f);
glColor3f(0.55f,          0.45f,          0.65f);
glVertex2f(          0.4f,          0.15f);
glColor3f(0.35f,          0.25f,          0.45f);
glVertex2f(          1.0f,          -0.2f);
glColor3f(0.55f,          0.45f,          0.65f);
glVertex2f( 1.0f,  0.25f);    glEnd();
    }

void drawGround() {
    glColor3f(0.15f, 0.5f, 0.2f);

```

```

        glBegin(GL_QUADS);
glVertex2f(-1.0f, -1.0f);
glVertex2f(-1.0f, -0.2f);
glVertex2f( 1.0f, -0.2f);
glVertex2f( 1.0f, -1.0f);    glEnd();
    }

void drawHorizonLine() {
glColor3f(1.0f, 1.0f, 1.0f);
glEnable(GL_LINE_STIPPLE);
glLineStipple(1, 0x00FF);
glLineWidth(2.0f);
glBegin(GL_LINE_STRIP);
glVertex2f(-1.0f, -0.2f);
glVertex2f( 1.0f, -0.2f);    glEnd();
glDisable(GL_LINE_STIPPLE);
}

void display() {
    glClearColor(0.6f, 0.75f, 0.95f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT);

    drawSun();
drawMountains();
drawGround();
drawHorizonLine();

    glFlush();
}

int main(int argc, char** argv) {
glutInit(&argc, argv);
glutInitWindowSize(700, 700);
    glutCreateWindow("Ex20 - Mini Scene: Primitives + Color");
glutDisplayFunc(display);    glutMainLoop();
    return 0;
}

```

Part B — Practice Exercises (Q01–Q20)

Attempt each exercise on your own first. Compile and run your own solution, compare its output to the task description, and only THEN check Part C. Difficulty is marked Easy, Medium, or Hard.

Easy Exercises

Exercise Q01 — Plus-Shaped Colored Points [Easy]

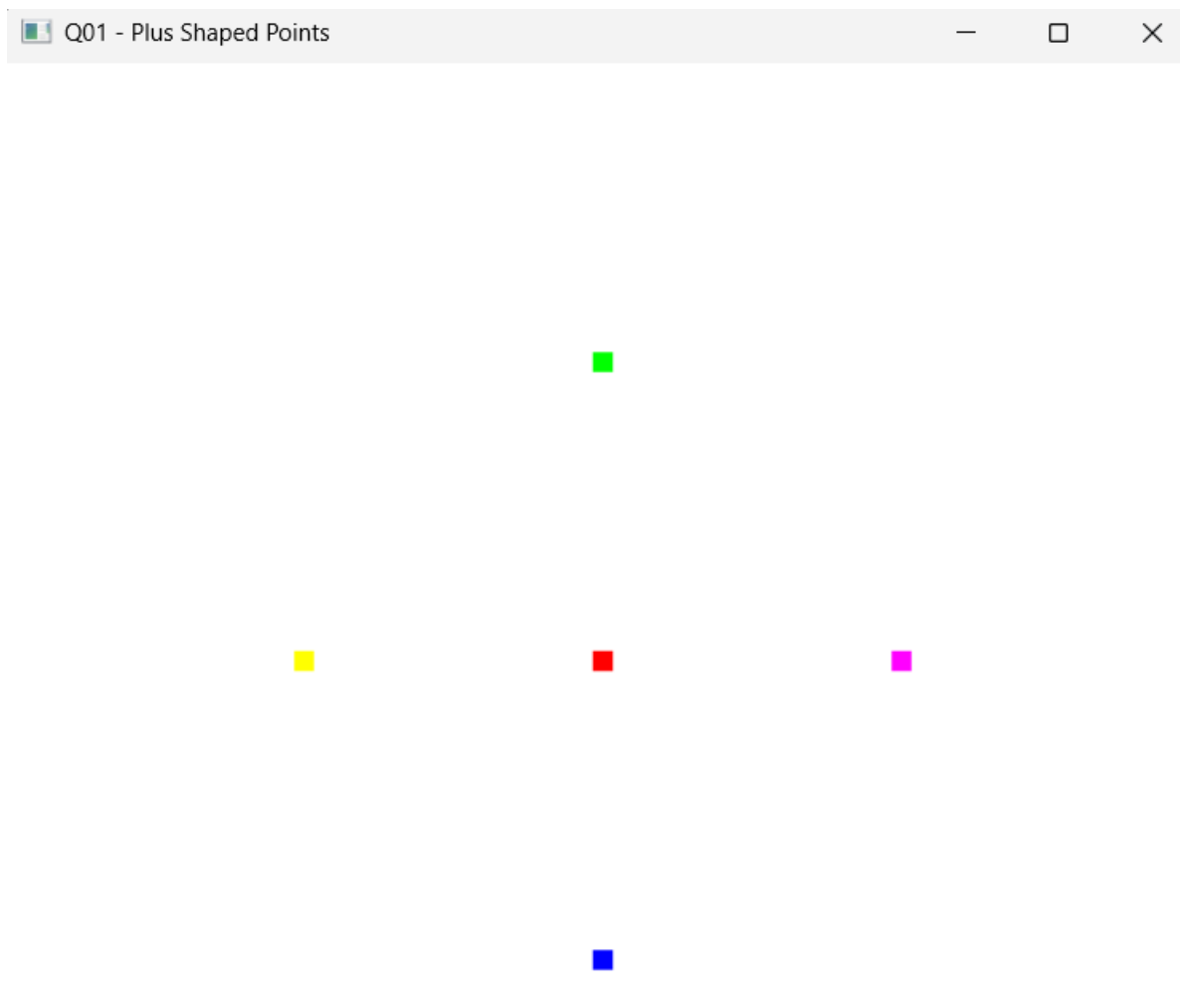
Objective: Practice GL_POINTS with a different color per vertex.

Task: Draw five individual points arranged in a plus (+) shape (center, up, down, left, right), each a DIFFERENT color, using GL_POINTS.

Hints:

- Set glPointSize large enough to see clearly (6-10).
- Call glColor3f immediately before each glVertex2f, inside the same glBegin/glEnd block.

Output:



Exercise Q02 — Two Unconnected Lines [Easy] Objective:

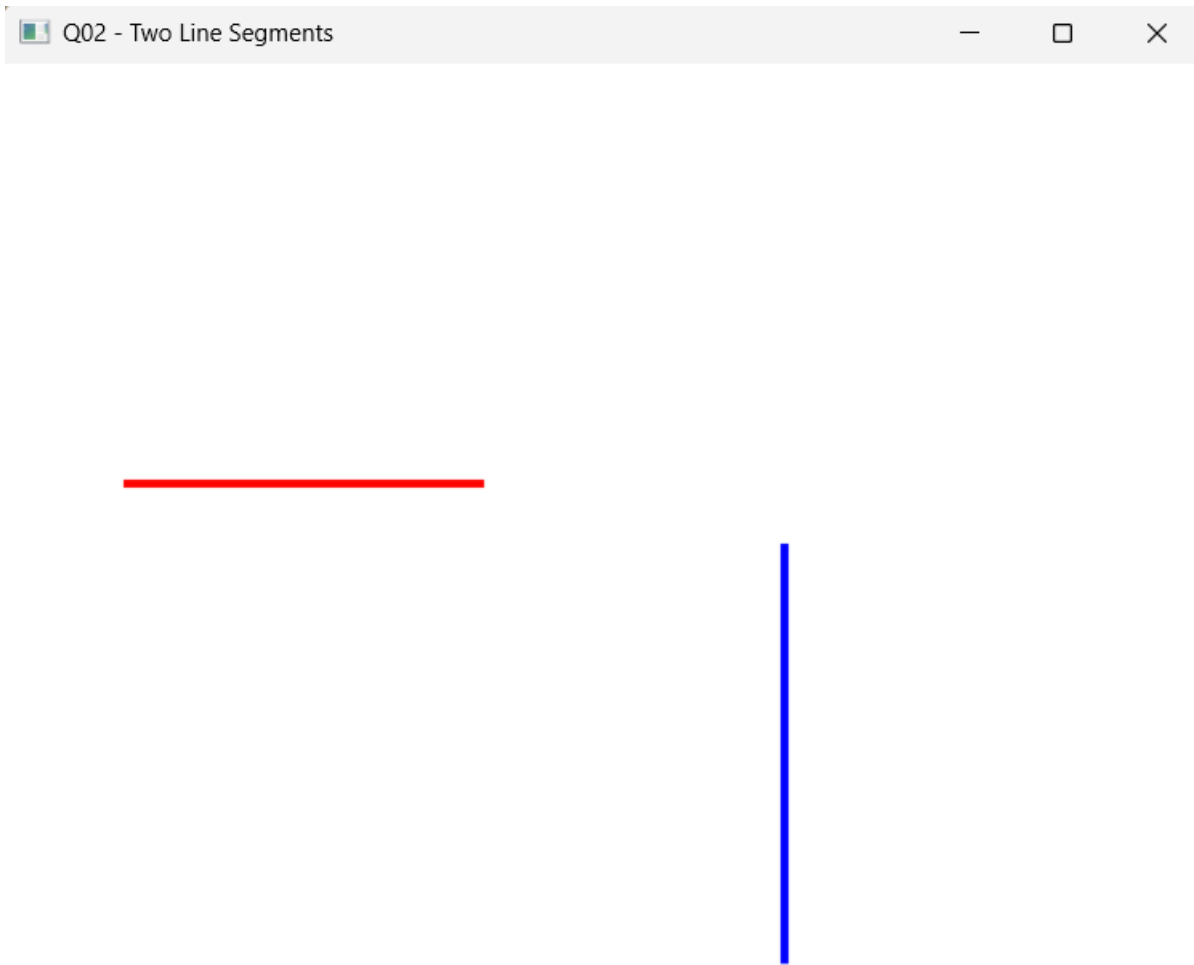
Practice GL_LINES vertex pairing.

Task: Draw two separate line segments (one horizontal, one vertical) that do NOT touch, using a single GL_LINES block with four vertices.

Hints:

- GL_LINES groups vertices in pairs: (v0,v1) is segment 1, (v2,v3) is segment 2.
- Make sure the two segments' coordinates don't overlap.

Output:



Exercise Q03 — Zig-Zag Line Strip [Easy]

Objective: Practice GL_LINE_STRIP and confirm the shape stays open.

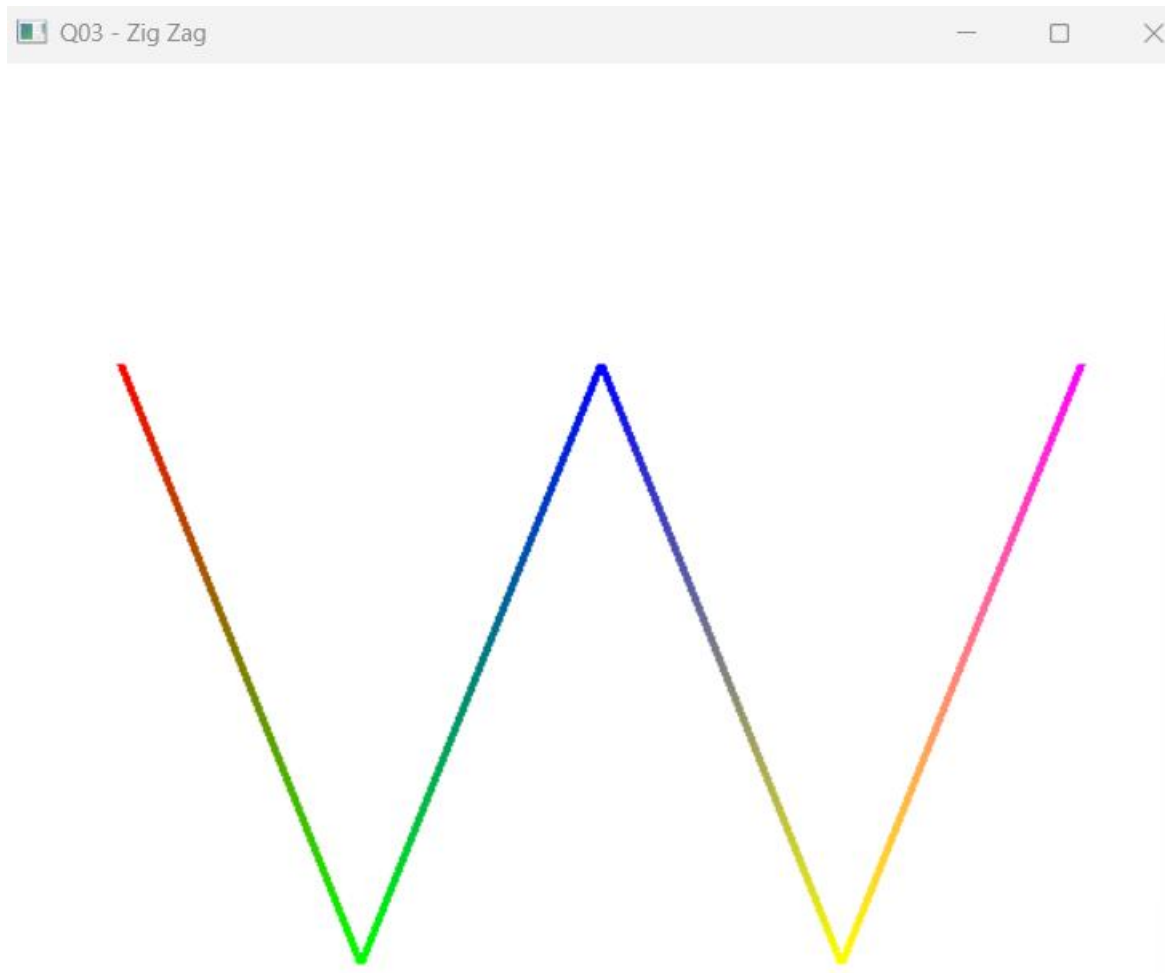
Task: Draw an open zig-zag pattern using at least 5 vertices with GL_LINE_STRIP. Do not let it close back to the start.

Hints:

- GL_LINE_STRIP connects consecutive vertices only — it never auto-closes.

- Alternate Y values up and down while X increases steadily for a zig-zag look.

Output:



Exercise Q04 — Square Outline (GL_LINE_LOOP) [Easy]

Objective: Practice GL_LINE_LOOP and glLineWidth together.

Task: Draw a closed square OUTLINE using GL_LINE_LOOP with a line width of 3.0.

Hints:

- Set glLineWidth(3.0f) BEFORE the glBegin call.
- Four vertices are enough — GL_LINE_LOOP automatically closes the shape.

Output:



Exercise Q05 — Dashed Line [Easy]

Objective: Practice the enable/set/disable pattern for line stippling.

Task: Draw a horizontal dashed line using `glLineStipple` with pattern `0x00FF` and `repeatFactor 1`.

Hints:

- Remember: `glEnable(GL_LINE_STIPPLE)` must come before drawing.
- Call `glDisable(GL_LINE_STIPPLE)` after `glEnd()` as good practice.

Output:



Exercise Q06 — Triangle with glColor3ub [Easy]

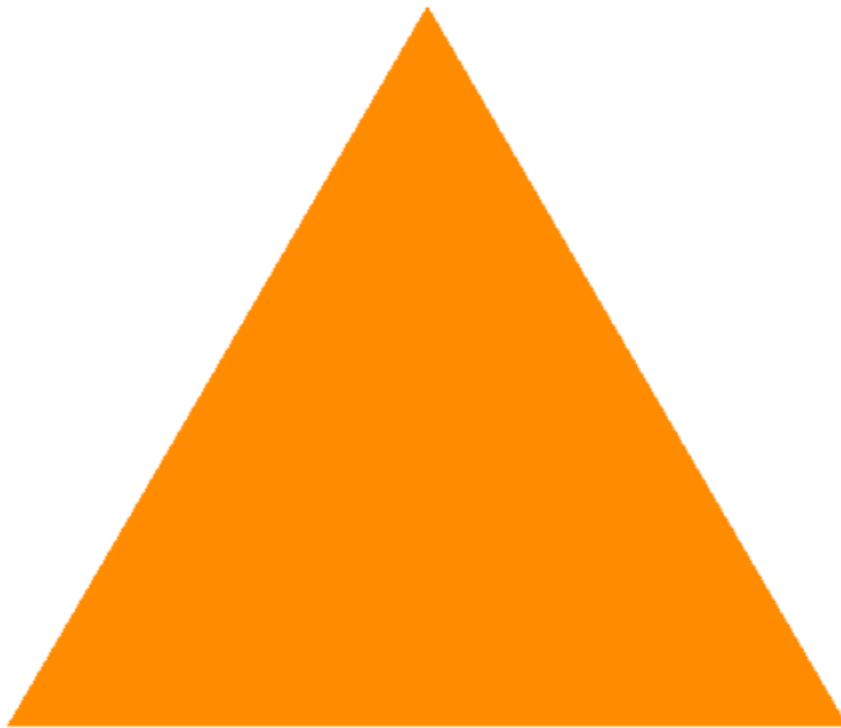
Objective: Practice specifying color with the 0-255 integer range instead of floats.

Task: Draw a filled triangle using glColor3ub (not glColor3f) with an orange color of your choosing.

Hints:

- glColor3ub takes three integers from 0 to 255, e.g. glColor3ub(255, 140, 0).
- Everything else about GL_TRIANGLES stays the same.

Output:



Exercise Q07 — One Quadrilateral (GL_QUADS) [Easy]

Objective: Practice specifying exactly four vertices for GL_QUADS.

Task: Draw one filled quadrilateral (it does not need to be a perfect rectangle) using GL_QUADS.

Hints:

- Any 4 points work as long as the shape stays convex (does not fold in on itself).
- List vertices in order, going around the shape (clockwise or counter-clockwise).

Output:



Medium Exercises

Exercise Q08 — Filled Hexagon (GL_POLYGON) [Medium]

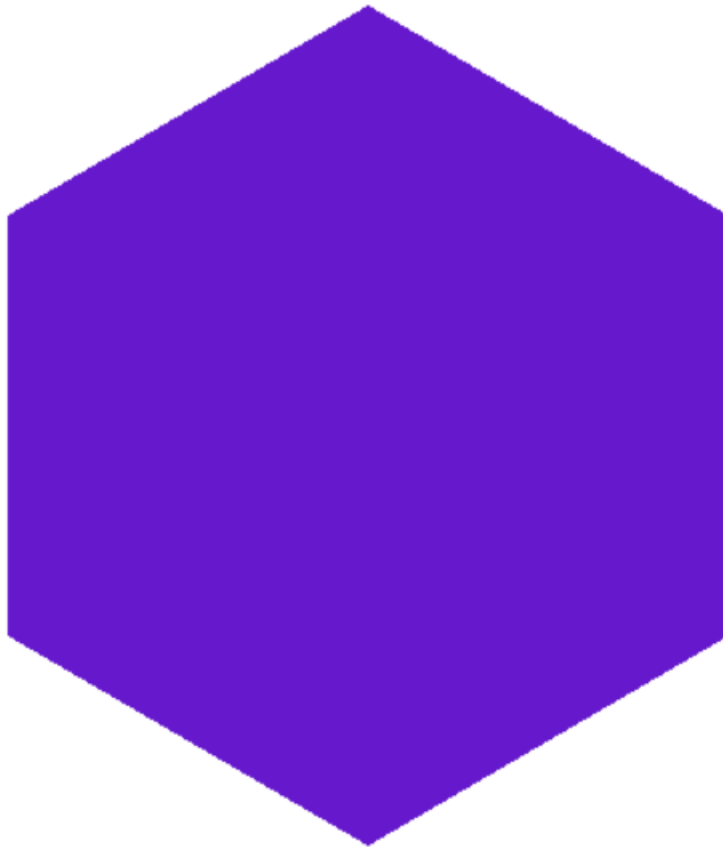
Objective: Practice typing an ordered vertex list for GL_POLYGON.

Task: Draw a filled, six-sided hexagon using GL_POLYGON, with your own choice of size, position, and color (different from the lecture example).

Hints:

- Plan your six vertices on paper first, going consistently clockwise or counter-clockwise.
- GL_POLYGON needs at least 3 vertices; fewer than that renders nothing.

Output:



Exercise Q09 — Rectangle from Triangle Strip [Medium]

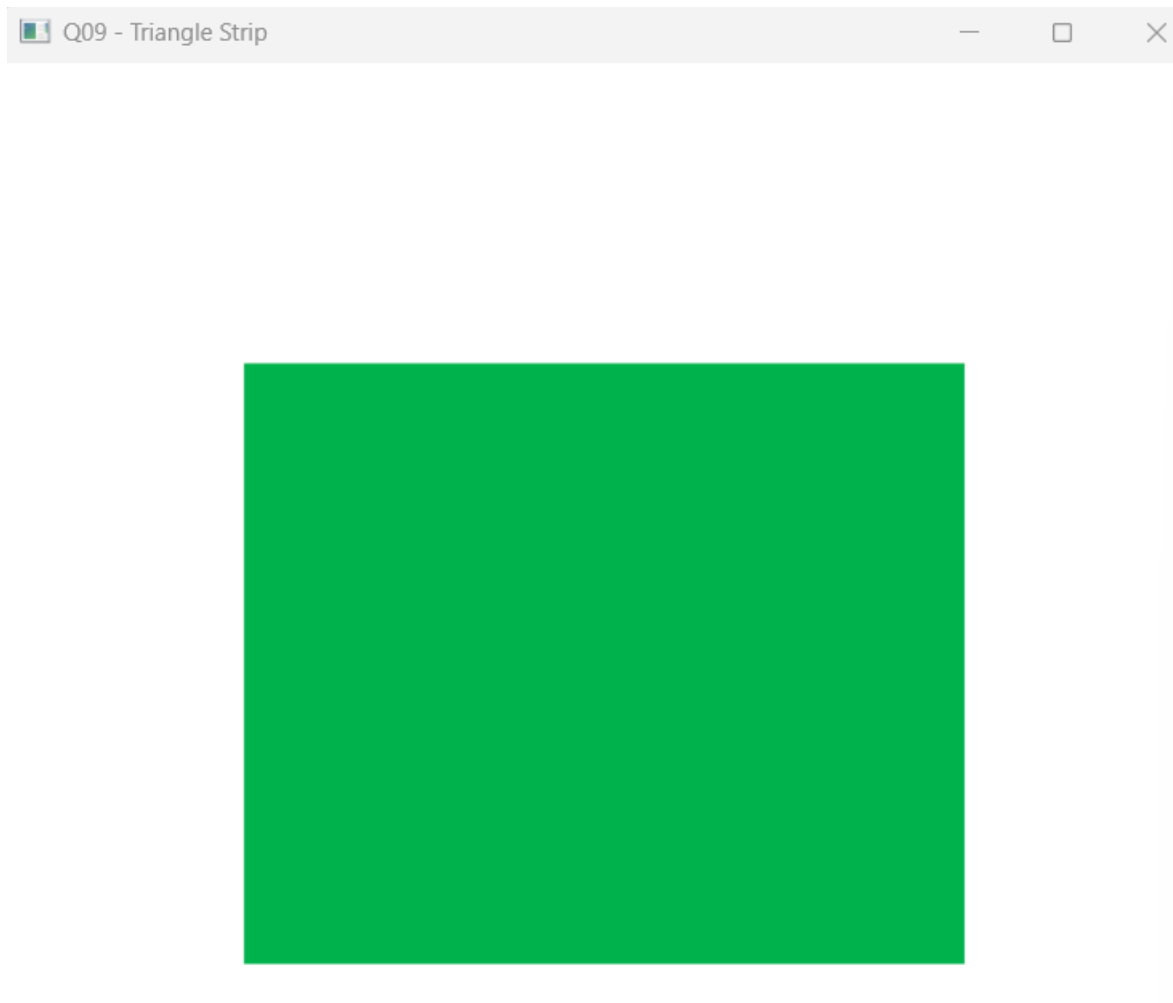
Objective: Practice GL_TRIANGLE_STRIP's shared-edge vertex rule.

Task: Use GL_TRIANGLE_STRIP with exactly 4 vertices to build a filled rectangle out of two triangles.

Hints:

- Order matters: v0-v1-v2 form the first triangle, v1-v2-v3 form the second.
- Try (left-bottom, left-top, right-bottom, right-top) as your four vertices.

Output:



Exercise Q10 — Half-Circle Fan [Medium]

Objective: Practice `GL_TRIANGLE_FAN` with a shared center vertex and trigonometry.

Task: Use `GL_TRIANGLE_FAN` to draw a half-circle 'pie' shape spanning 180 degrees, using 6-8 outer vertices plus the shared center.

Hints:

- The first vertex in the `glBegin` block is the shared center (the 'point' of the fan).
- Sweep your angle loop from 0 to π radians (not the full 2π) for a half circle.

Output:



Exercise Q11 — Two Quads, One glBegin Block [Medium]

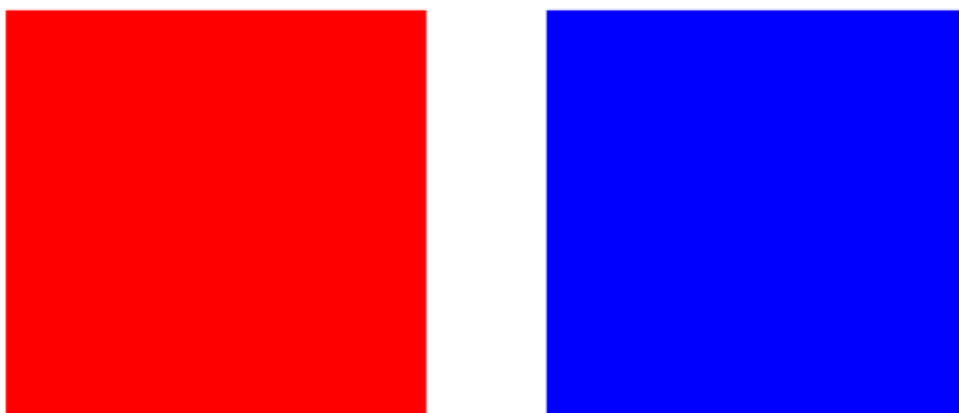
Objective: Practice multiple independent quads inside a single `GL_QUADS` block, each visually distinct.

Task: Draw two separate quads (8 vertices total) in ONE `glBegin(GL_QUADS)` block, making each quad a clearly different color using per-vertex `glColor3f` calls.

Hints:

- `GL_QUADS` groups vertices in fours: the first four form quad 1, the next four form quad 2.
- Set the color right before each quad's first vertex — it will apply to that whole group.

Output:



Exercise Q12 — Staircase Ribbon (GL_QUAD_STRIP) [Medium]

Objective: Practice the alternating bottom/top vertex order required by GL_QUAD_STRIP.

Task: Use GL_QUAD_STRIP to build a 'staircase' ribbon made of at least 3 connected quads of increasing height.

Hints:

- Remember: vertices alternate bottom, top, bottom, top as you move across the strip (NOT outline order).
- Increase the 'top' Y coordinate for each successive pair to create a rising staircase look.

Output:



Exercise Q13 — Yellow-to-Purple Gradient Line [Medium]

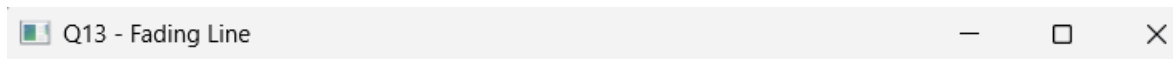
Objective: Practice per-vertex color interpolation on a simple `GL_LINES` primitive.

Task: Draw a single thick line that fades from yellow at one end to purple at the other, using per-vertex color.

Hints:

- Set one color right before the FIRST vertex, and a different color right before the SECOND vertex, both inside the same `glBegin/glEnd` block.
- Use a large `glLineWidth` so the gradient is easy to see.

Output:



Exercise Q14 — Two Different Stipple Patterns [Medium]

Objective: Practice managing `GL_LINE_STIPPLE` state across multiple shapes.

Task: Draw two lines in the same window, each using a DIFFERENT stipple pattern and/or repeatFactor. Enable and disable stippling around each line separately.

Hints:

- Wrap each line's own `glEnable/glLineStipple/glBegin.../glEnd/glDisable` in its own group — don't just set the pattern once and hope it carries over correctly.
- Try comparing `0x00FF` (dashed) against `0xAAAA` (dotted).

Output:



Hard Exercises

Exercise Q15 — Smoothly Shaded Square [Hard]

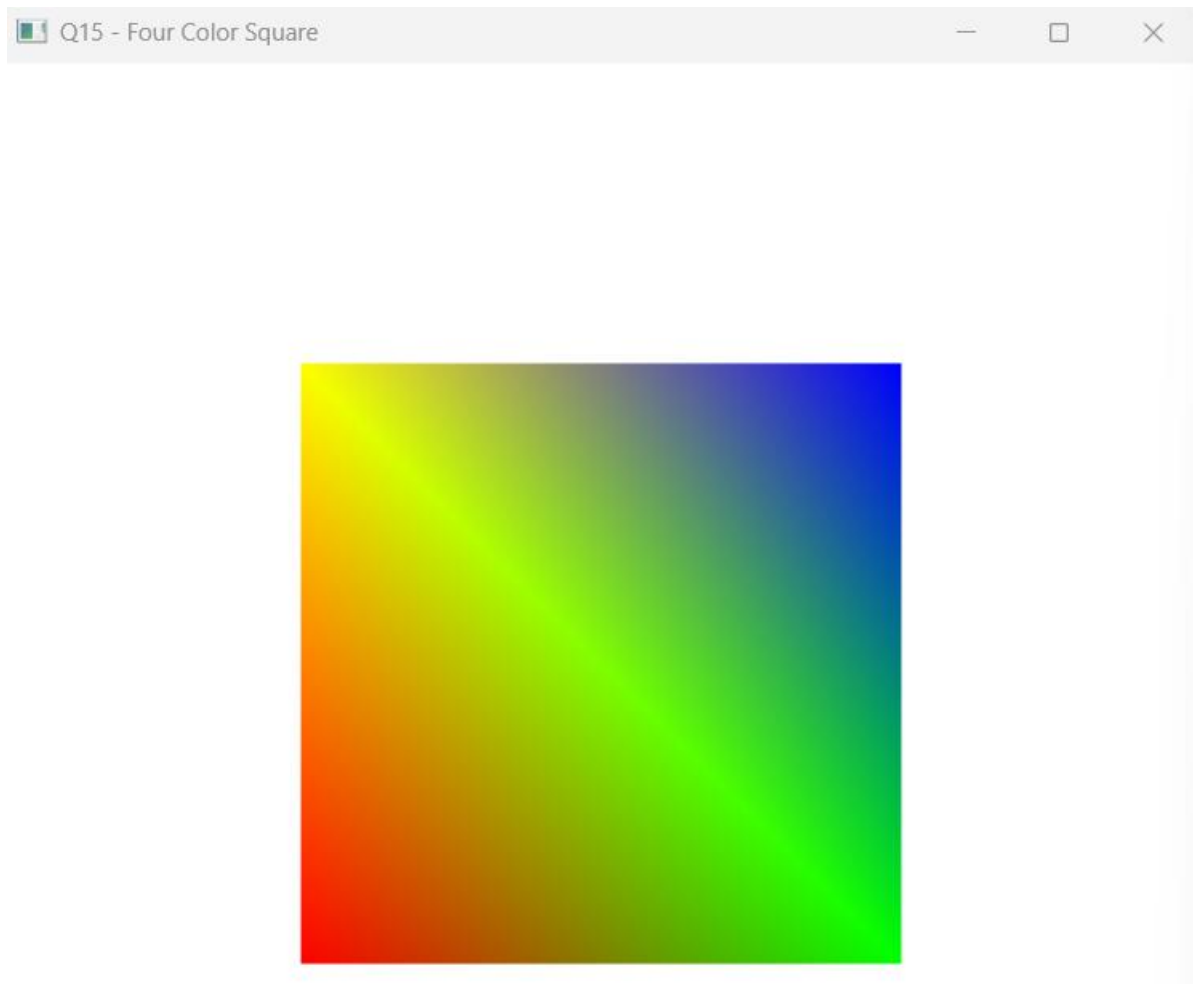
Objective: Practice per-vertex color interpolation across a 4-vertex filled primitive.

Task: Draw a filled square using `GL_QUADS` where all FOUR vertices have different colors, producing a smoothly blended interior.

Hints:

- Use four clearly different colors (e.g. red, green, blue, yellow) — one per corner.
- Compare the result to Example 18's triangle version — notice how the blending works with four colors instead of three.

Output:



Exercise Q16 — Fan + Quad Composition [Hard]

Objective: Practice combining two different primitive types into one composite object.

Task: Build a simple 'traffic cone' or 'ice cream cone' shape: a colorful circle using `GL_TRIANGLE_FAN` for the top, and a `GL_QUADS` rectangle for the base, using clearly different colors for each part.

Hints:

- Write a separate helper function for each part (e.g. `drawCircle()`, `drawBase()`) and call both from `display()`.
- Position the circle and the rectangle so they visually connect (e.g. circle sits just above the base).

Output:



Exercise Q17 — Toggle Stipple with Keyboard [Hard]

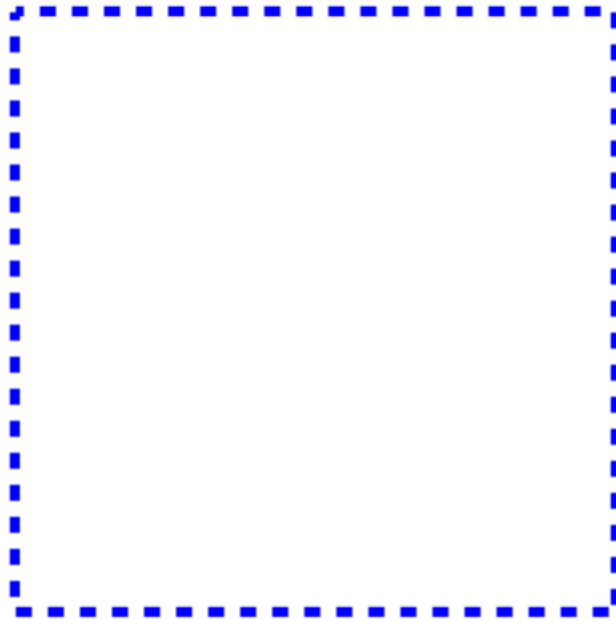
Objective: Practice combining `glutKeyboardFunc` with conditional stippling state.

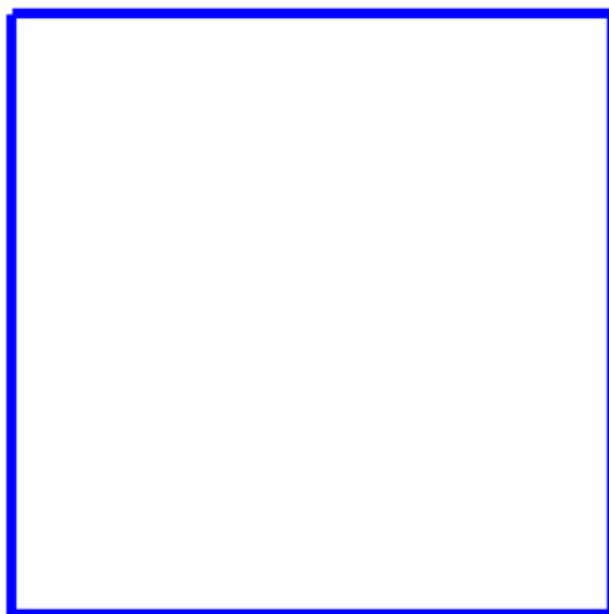
Task: Draw a square outline. Let the user press the 's' key to toggle `GL_LINE_STIPPLE` on and off, redrawing the outline each time.

Hints:

- Store a bool `stippleOn` as a global variable; flip it inside your keyboard callback and call `glutPostRedisplay()`.
- In `display()`, only call `glEnable/glLineStipple/glDisable` when `stippleOn` is true.

Output:





Exercise Q18 — Alternating-Color Ribbon [Hard]

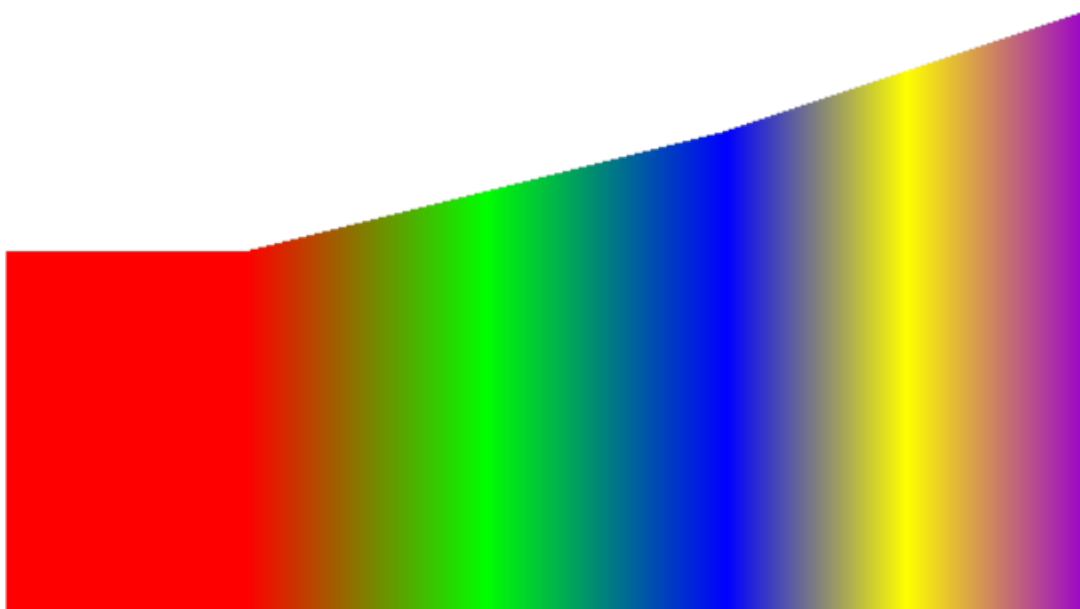
Objective: Practice per-pair (rather than per-vertex-blend) coloring across a `GL_QUAD_STRIP`.

Task: Build a ribbon of at least 5 connected quads using `GL_QUAD_STRIP`, where each quad has a CLEARLY different solid color from its neighbors (not a smooth blend).

Hints:

- Set the color once per vertex PAIR (bottom+top) rather than smoothly varying it, so each quad segment reads as one solid color.
- A loop that alternates between two colors based on the segment index works well.

Output:



Exercise Q19 — Overlapping Alpha-Blended Quads [Hard]

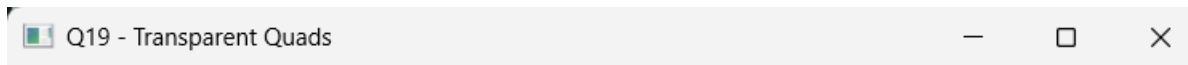
Objective: Practice glColor4f with GL_BLEND to create real transparency.

Task: Draw two overlapping quads, each a different color with alpha around 0.5, using glEnable(GL_BLEND) so the overlapping region visibly shows both colors mixed.

Hints:

- Don't forget glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA) alongside glEnable(GL_BLEND).
- Make sure the two quads' coordinate ranges actually overlap, or you won't see any blending.

Output:



Exercise Q20 — Procedural Rainbow Fan (Capstone) [Hard]

Objective: Practice procedural color generation combined with `GL_TRIANGLE_FAN` and a configurable loop count.

Task: Draw a circular `GL_TRIANGLE_FAN` whose outer vertices smoothly cycle through rainbow colors as you go around, using a named constant such as `SEGMENT_COUNT` to control how many slices are generated (do not hardcode each vertex/color by hand).

Hints:

- Write a helper function that converts a fraction t (0 to 1) into an RGB color, e.g. using `sin()` with three different phase offsets for red/green/blue.
- Loop i from 0 to `SEGMENT_COUNT`, compute $t = i / \text{SEGMENT_COUNT}$ and $\text{angle} = t * 2 * \text{PI}$, then set the color and vertex together each iteration.

Output:

